

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 3 OF 9

TECHNICAL ARCHITECTURE

Technology Stack - Coding Standards - Folder Structure - Logging - Caching - Queueing

Field	Value
Document Status	Draft - v0.1 - Phase 3 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 1 Business Architecture; Vol. 2 Application Architecture (16 bounded contexts CTX-01...CTX-16, module dependency graph, event catalog)
Downstream Volumes	Vol. 4 Data Architecture; Vol. 5 Integration Architecture; Vol. 6 Security Architecture; Vol. 7 Infrastructure & Deployment; Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume selects and justifies the concrete technology stack, and defines the engineering conventions (folder structure, coding standards, logging, caching, queueing, multi-tenancy implementation) needed to start writing code against the bounded contexts fixed in Volume 2. Every decision below is written as a lightweight ADR (Architecture Decision Record): context, decision, and consequences, so it can be revisited explicitly rather than silently drifted from.

1.2 Constraints Inherited From Earlier Volumes

- Vol. 1 Principle P1: multi-tenancy is first-class - every table, cache key, queue message, and log line carries a tenant identifier (Section 6, 7, 8, 11 of this volume implement this).
- Vol. 1 Principle P3/P4: the ledger is the single source of truth and is append-only/auditable - this constrains the database and ORM choices in Section 2.
- Vol. 2 Section 2 decision: modular monolith first, with enforced module boundaries - this constrains the folder structure (Section 3) and coding standards (Section 4).
- Vol. 2 Section 10 event catalog: the platform needs a durable, ordered-per-key event bus - this constrains Section 8's queueing technology choice.

2. Technology Stack Selection

Each decision below follows the same format: Context, Options Considered, Decision, Rationale, Consequences. Where the choice affects cost or ops complexity, that trade-off is stated explicitly rather than assumed.

2.1 Backend Language & Framework

Field	Detail
Options Considered	TypeScript/NestJS; Java/Spring Boot; C#/.NET; Python/Django
Decision	TypeScript on Node.js, using NestJS as the application framework
Rationale	NestJS's module system maps directly onto Vol. 2's bounded-context boundaries (each CTX-xx becomes a NestJS module with an explicit exported provider surface, enforcing "no cross-module direct database access"). TypeScript end-to-end (backend + frontend, Section 2.3) lets domain types (Vol. 2 Section 8 UML) be shared as a single source of truth via a shared types package, eliminating an entire class of integration bugs. NestJS has first-class support for dependency injection, decorators-based validation, and modular testing, all of which Section 4/Vol. 8 rely on.
Consequences	Team must standardize on Node.js performance patterns for CPU-heavy work (e.g. large report generation, Section 9) - offload those to worker processes/queues rather than blocking the main event loop. Node's single-threaded model is not a concern for typical I/O-bound accounting operations.

2.2 Primary Datastore

Field	Detail
Options Considered	PostgreSQL; MySQL; SQL Server; MongoDB (document store)
Decision	PostgreSQL
Rationale	Financial data is inherently relational with strict integrity constraints (Vol. 2 Section 8 double-entry model); PostgreSQL's transactional guarantees, mature JSON column support (for custom fields, Vol. 1 BC-17), Row-Level Security (Section 11 multi-tenancy), and extension ecosystem (e.g. pg_partman for partitioning large ledger tables) make it the strongest fit. A document store was rejected because financial reporting requires strong joins and aggregate consistency that document stores handle poorly at this integrity bar.
Consequences	Vol. 4 must design partitioning/indexing strategy for high-volume tables (Journalline, InvoiceLineItem) from day one, since these grow unbounded per tenant.

2.3 Frontend Framework

Field	Detail
Options Considered	React; Vue; Angular
Decision	React with TypeScript, using a component library built on a headless UI kit (e.g. Radix primitives) rather than a heavyweight admin template
Rationale	Largest talent pool and ecosystem; shares TypeScript types with the NestJS backend (Section 2.1); React Query (or equivalent) gives a clean pattern for the API-first architecture (Vol. 2 Principle: UI is a client of the same API third parties use).
Consequences	Vol. 9 (UI/UX Design Specification) must define the component library and design tokens

Field	Detail
	before feature teams start building screens, to avoid inconsistent UI drift across the ~16 bounded-context-aligned feature areas.

2.4 Cache Layer

Field	Detail
Decision	Redis (or Redis-compatible, e.g. managed ElastiCache/Memorystore)
Rationale	Industry-standard, supports the cache patterns in Section 7 (read-through caching, rate limiting, session storage, distributed locks for e.g. invoice numbering sequences), and is a dependency most cloud providers offer managed.

2.5 Event Bus / Message Broker

Field	Detail
Options Considered	Apache Kafka; RabbitMQ; cloud-native (SQS/SNS, Pub/Sub)
Decision	Apache Kafka (or managed equivalent, e.g. Confluent Cloud/MSK) for the domain event bus (Vol. 2 Section 10); RabbitMQ-style task queues are used underneath for simple background jobs (Section 9) where Kafka's log-retention model is unnecessary overhead
Rationale	Financial domain events (JournalPosted, InvoiceSent) benefit from Kafka's durable, replayable log - this gives natural audit trail redundancy (Vol. 1 Principle P4) and lets a new consumer (e.g. a future Reporting read-model rebuild) replay history rather than requiring a backfill migration. Per-key ordering (partitioned by tenant + aggregate ID) preserves causal order of events for a given invoice or journal entry.
Consequences	Requires operating a Kafka cluster (or paying for a managed one) from Release 1 even while the application itself is a single monolith process - this is accepted because event-driven decoupling (Vol. 2 Section 2.2) is needed from day one, not just after service extraction.

2.6 Search & Reporting Index

Field	Detail
Decision	OpenSearch (Elasticsearch-compatible), fed by the Kafka event stream, used for the Reporting & Analytics module's (CTX-11) full-text and aggregate query needs
Rationale	PostgreSQL read replicas alone are insufficient for the custom-report-builder and cross-module aggregate queries in Vol. 1 BC-13; a denormalized, event-sourced search index keeps Reporting's read-only, downstream position (Vol. 2 Section 3.3) without adding query load to the transactional database.

2.7 Object/File Storage

Field	Detail
Decision	S3-compatible object storage (AWS S3 or self-hosted MinIO for on-prem/dev), used by the Documents context (CTX-12) for attachments and by Reporting for exported PDFs/spreadsheets

2.8 API Style

Field	Detail
Decision	REST/JSON as the primary public and internal API style (Vol. 9 to formalize), with webhooks for outbound event notification to third parties (Vol. 1 BC-19). GraphQL is deferred - not adopted at Release 1.
Rationale	REST is what most accounting-software integrators (Zoho, QuickBooks, Xero ecosystems) expect; a GraphQL BFF can be layered in later for the web app specifically without changing the underlying service contracts, if UI query flexibility becomes a bottleneck.

3. Repository & Folder Structure

3.1 Repository Strategy Decision

Field	Detail
Decision	Monorepo (single repository) for Release 1, using a build-orchestration tool (e.g. Nx or Turborepo) to manage the module boundaries as enforceable project graph constraints
Rationale	The modular monolith decision (Vol. 2 Section 2.1) means all 16 bounded contexts deploy together at Release 1 - a monorepo keeps cross-module refactors atomic and lets the shared-types package (Section 2.3) be consumed without a package-publish step. Nx/Turborepo's project-graph lint rules directly enforce Vol. 2 Section 7's "no upward dependency" layering.
Extraction path	When a module is extracted to its own deployable (Vol. 2 Section 2.1 extraction triggers), it can be moved to its own repository without restructuring its internal code, since its folder is already a self-contained Nx/Turborepo project.

3.2 Top-Level Folder Structure

Folder tree (abbreviated)

```

/ledger-platform
/apps
  /web-app          # React SPA (business-owner/staff facing)
  /portal-app       # React SPA (customer/vendor self-service, CTX-13)
  /core-api         # NestJS host process (modular monolith, Release 1)
  /payments-service # Extracted service (CTX-06) - separate deploy from Release 1
  /integration-gateway # Extracted service (CTX-16) - separate deploy from Release 1
  /workers          # Background job / queue consumers (Section 9)
/libs
/modules
  /identity-tenant  # CTX-01
  /ledger           # CTX-02
  /receivables      # CTX-03
  /payables         # CTX-04
  /banking          # CTX-05
  /inventory        # CTX-07
  /projects         # CTX-08
  /payroll          # CTX-09 (R3)
  /tax-compliance   # CTX-10
  /reporting-analytics # CTX-11
  /documents        # CTX-12
  /collaboration-portals # CTX-13
  /automation-customization # CTX-14
  /ai-assistant     # CTX-15 (R2/R3)
/shared
  /domain-types     # Shared TypeScript types/DTOs (Vol.2 Sec.8 UML)
  /event-contracts  # Kafka event schemas (Vol.2 Sec.10 catalog)
  /ui-components    # Shared React component library (Vol.9)
  /testing-utils    # Shared test fixtures/mocks (Vol.8)
/infra              # IaC, CI/CD pipeline definitions (Vol.7)
/docs               # This blueprint and derived documents

```

3.3 Inside a Module (e.g. /libs/modules/receivables)

Per-module internal layering (maps to Vol.2 Sec.6 component diagrams)

```
/receivables
/api          # Controllers / route handlers - Invoice, Quote, SalesOrder endpoints
/domain      # Domain services, aggregates, business rules (invoice lifecycle, Vol.2 Sec.8.1 state
machine)
/infrastructure # Repository implementations (Postgres), event publisher (Kafka)
/events      # Inbound event handlers (subscribes to InventoryStockChanged, etc.)
/dto         # Request/response DTOs (validated via class-validator)
/tests       # Unit + integration tests for this module only
index.ts     # Public export surface - the ONLY thing other modules may import
```

The index.ts export-surface rule is the enforcement mechanism for Vol. 2's "no cross-module direct database access": a lint rule forbids any import path that reaches past a module's index.ts into its internal folders (e.g. forbidding import ... from '@ledger-platform/receivables/infrastructure/repository' from outside the receivables module).

4. Coding Standards

4.1 Enforcement Mechanism

Standards below are enforced automatically wherever possible (linter rules, CI gate) rather than relying on code review discipline alone. Vol. 8 (Testing & QA) defines the CI pipeline gates that run these checks on every pull request.

4.2 Core Rules

#	Rule	Enforcement
CS-01	A module may only import another module's index.ts export surface, never its internals	ESLint import-boundary rule + Nx module-boundary lint
CS-02	A module may only depend on modules in a strictly lower architecture layer (Vol. 2 Section 7)	Nx project-graph constraint (tags: layer:0...layer:4)
CS-03	All financial amounts use a Decimal/fixed-point type, never native floating point	Code review + typed wrapper class (Money) required by domain type definitions
CS-04	All database writes to financial entities go through the domain service layer, never direct repository calls from a controller	Architecture test (dependency-cruiser or similar) run in CI
CS-05	Every public API endpoint has an OpenAPI schema and a corresponding integration test	CI gate blocks merge if OpenAPI spec or test is missing
CS-06	Every domain event has a versioned schema in /libs/shared/event-contracts; breaking changes require a new version, not a mutation	Schema registry validation in CI
CS-07	No module-level global mutable state; all cross-request state flows through request context (includes tenant ID, user ID, correlation ID - Section 6)	Code review checklist + lint rule against module-scope `let`/mutable exports
CS-08	Naming: REST resources are plural nouns (/invoices), domain events are past-tense (InvoiceSent), commands are imperative (SendInvoice)	Code review checklist

4.3 Error Handling Convention

Standard error shape (all modules)

```
{
  "error": {
    "code": "RECEIVABLES.INVOICE_ALREADY_PAID",
    "message": "Invoice INV-0042 is already fully paid and cannot be voided.",
    "correlationId": "c7f1e2...",
    "details": { "invoiceId": "...", "currentStatus": "Paid" }
  }
}
```

Error codes are namespaced by bounded context (RECEIVABLES., PAYABLES., LEDGER., ...) so client applications and support tooling can route/handle errors predictably. Every error response carries the same correlationId as the structured log entry for that request (Section 6), so a support engineer can go from a user-reported error code straight to the relevant log lines.

5. API Design Conventions (Summary)

Full endpoint-by-endpoint contracts belong to Volume 9; this section fixes the conventions every endpoint must follow so Volume 9 does not have to re-derive them per bounded context.

Convention	Rule
Versioning	URL-path versioned: /v1/invoices. A new major version is only introduced for breaking changes; additive fields never bump the version.
Pagination	Cursor-based (?cursor=...&limit=50), not offset-based, to stay stable under concurrent writes to large tables (e.g. JournalLine).
Filtering/Sorting	Consistent query-param syntax: ?filter[status]=Sent&sort=-createdAt across all bounded-context APIs.
Idempotency	All POST endpoints that create a financial transaction accept an Idempotency-Key header, checked against a Redis-backed dedup store (Section 7) before processing.
Auth	Bearer JWT (short-lived) issued by CTX-01 Identity & Tenant, containing tenant ID and role claims, validated by every module's API layer via shared middleware.
Tenant scoping	Every request is scoped to a tenant from the JWT claim, never from a client-supplied tenant ID parameter, to prevent tenant-spoofing (Vol. 6 will formalize this as a security control).

6. Logging Strategy

6.1 Principles

- Structured JSON logging only - no unstructured string logs - so logs are queryable in the observability platform (Vol. 7 will select the concrete tool, e.g. an ELK/OpenSearch or vendor SaaS stack).
- Every log line carries: tenantId, userId (if applicable), correlationId, module (CTX-xx), and a severity level.
- Correlation ID is generated at the API Gateway (Vol. 2 Section 5) on every inbound request and propagated through every internal call and published event, so one user action can be traced end-to-end across modules even inside the monolith.
- Financial-mutation log lines (ledger postings, invoice status changes) are never sampled or dropped - they log at INFO level unconditionally, distinct from high-volume debug logs which may be sampled under load.

6.2 Log Levels

Level	Usage
ERROR	Unhandled exceptions, failed financial postings, failed external integration calls that could not be retried successfully.
WARN	Retried operations, degraded external dependency (e.g. bank feed provider slow), approaching rate limits.
INFO	Business-significant events: invoice sent, payment recorded, journal posted, user login. Always includes the correlationId and tenantId.
DEBUG	Detailed request/response payloads, cache hit/miss - disabled in production by default, enabled per-tenant or per-request for support investigations.

Example structured log line

```
{
  "timestamp": "2026-07-13T10:22:31.104Z",
  "level": "INFO",
  "module": "CTX-03-receivables",
  "event": "invoice.sent",
  "tenantId": "org_8f2a1c",
  "userId": "usr_4471",
  "correlationId": "c7f1e2-9b3a-4d10",
  "message": "Invoice INV-0042 sent to customer",
  "context": { "invoiceId": "inv_0042", "amount": 1250.00, "currency": "USD" }
}
```

7. Caching Strategy

7.1 Cache Layers

Layer	Technology	Used For	Invalidation
Application read cache	Redis	Frequently-read, rarely-changed reference data: chart of accounts, tax rate tables, item master, customer list	Write-through: module publishes cache-invalidation on its own domain event (e.g. Inventory invalidates item cache on StockAdjusted)
Computed/aggregate cache	Redis	Dashboard summaries, report snapshots (Vol.1 BC-13)	Time-based TTL (e.g. 5 min) plus explicit invalidation on relevant JournalPosted/InvoiceSent events
Session/auth cache	Redis	JWT refresh-token state, active session list, rate-limit counters	TTL matching token lifetime
Idempotency store	Redis	Idempotency-Key deduplication (Section 5) for financial-mutating POST requests	TTL of 24h, matching the client retry window
Distributed lock	Redis (Redlock pattern)	Sequential number generation (invoice numbers, journal entry numbers) to avoid duplicate numbers under concurrent requests	Lock released immediately after number is reserved

7.2 Cache Key Convention (Tenant-Safe)

Cache key pattern - tenant ID is always the first segment

```
{tenantId}:{module}:{entity}:{entityId}
example: org_8f2a1c:receivables:invoice:inv_0042
example: org_8f2a1c:ledger:coa:full
```

No cache key is ever constructed without a tenant ID prefix - this is a CI-enforced lint rule (Section 4) to prevent the single highest-risk multi-tenancy bug class: cross-tenant cache leakage. A shared cache utility wraps all Redis access so individual modules cannot bypass the key convention.

8. Queue & Asynchronous Processing Strategy

8.1 Event Bus Topic Design

Kafka topics are partitioned by tenantId + aggregateId to guarantee per-aggregate ordering (e.g. all events for one invoice arrive in order) while allowing horizontal consumer scaling across tenants.

Topic	Partition Key	Producers	Consumers
ledger.journal-events	tenantId+journalEntryId	Ledger (CTX-02)	Reporting, AI Assistant
receivables.invoice-events	tenantId+invoiceId	Receivables (CTX-03)	Ledger, Reporting, Collaboration, Automation
payables.bill-events	tenantId+billId	Payables (CTX-04)	Ledger, Reporting, Inventory
inventory.stock-events	tenantId+itemId	Inventory (CTX-07)	Receivables, Payables, Reporting
banking.transaction-events	tenantId+bankAccountId	Integration Gateway, Banking (CTX-05)	Receivables, Payables, Ledger
tenant.lifecycle-events	tenantId	Identity & Tenant (CTX-01)	All modules (initialization/deprovisioning)

8.2 Delivery Semantics & Idempotent Consumers

- At-least-once delivery is assumed for all Kafka consumers (exactly-once is not guaranteed end-to-end across this stack). Every consumer must be idempotent - reprocessing the same event must not double-post a journal entry or double-send a notification.
- Idempotency is achieved by consumers tracking a processed-event-id ledger (keyed by event ID, checked before applying side effects) rather than relying on the broker alone.
- Poison messages (repeatedly failing to process) are routed to a dead-letter topic after a bounded retry count (e.g. 5 attempts with exponential backoff) and surfaced as an operational alert (Vol. 7).

8.3 Background Jobs vs. Event-Driven Reactions

Pattern	When to Use	Example
Event-driven reaction (Kafka consumer)	Another module needs to react to something that already happened	Ledger reacting to InvoiceSent to post revenue
Scheduled background job (queue + cron trigger)	Work that happens on a time schedule, not in reaction to a user action	Recurring invoice generation (Vol.1 BC-16), overdue-invoice reminder emails, nightly report pre-computation
On-demand async job (queue, triggered by user action but too slow to run synchronously)	User-triggered work that would block the HTTP request if run synchronously	Bulk CSV import of items/customers, large report export to PDF/XLSX, bank statement OCR/auto-scan

9. Background Jobs & Scheduling

Scheduling architecture

```

flowchart LR
    Cron[Cron Scheduler - e.g. BullMQ repeatable jobs] --> Queue[(Redis-backed Job Queue)]
    UserAction[User-triggered async request] --> Queue
    Queue --> WorkerPool[Worker Process Pool - /apps/workers]
    WorkerPool --> Modules[Calls into module domain services]
    WorkerPool --> DLQ[(Dead Letter Queue on repeated failure)]
    WorkerPool --> Notify[Notify user/tenant on completion or failure]
  
```

Job	Trigger	Owning Module	Notes
Recurring invoice generation	Daily cron, per-tenant recurrence rule	Automation (CTX-14) -> Receivables	Must be idempotent per (tenant, recurrence rule, period) to survive worker retries
Overdue invoice reminders	Daily cron	Automation -> Receivables, Collaboration	Respects tenant-configured reminder cadence
Bank feed polling / import	Scheduled poll (e.g. every 4h) or provider webhook	Integration Gateway -> Banking	Webhook preferred where the provider supports it, to reduce latency and poll cost
Report pre-computation	Nightly + on-demand	Reporting (CTX-11)	Writes to OpenSearch index (Section 2.6), not the transactional DB
Payroll batch run	User-triggered, scheduled per pay period (R3)	Payroll (CTX-09)	Long-running; must support partial-failure recovery (some employees processed, one fails)
Depreciation posting	Monthly cron	Ledger (CTX-02) - Fixed Assets sub-capability	Posts journal entries automatically per asset depreciation schedule

10. Configuration & Feature Flag Management

Implements the entitlement-check mechanism anticipated by Vol. 1 Section 9 (Standard/Professional/Enterprise editions) and Vol. 2 Section 12's forward contract.

Config Type	Mechanism	Example
Edition entitlement	Per-tenant feature-flag set resolved at request time from the tenant record (CTX-01), cached in Redis (Section 7)	canUseInventory, canUsePayroll, maxCustomRoles
Environment configuration	Environment variables + a typed config-validation layer at boot (fails fast on missing/invalid config)	Database URL, Kafka brokers, external API keys
Operational feature flags	Flag-management service (e.g. a lightweight in-house table or a vendor tool) for gradual rollout independent of edition	Enabling the AI Assistant (CTX-15) for a beta cohort before general release
Tenant-level customization config	Stored per-tenant in the Identity & Tenant module (CTX-01), read by Automation/Customization (CTX-14)	Custom fields, custom report permissions, workflow rules

11. Multi-Tenancy Implementation Strategy

11.1 Decision

Field	Detail
Options Considered	(a) Database-per-tenant, (b) Schema-per-tenant within a shared PostgreSQL instance, (c) Shared schema with a tenant_id column on every table plus PostgreSQL Row-Level Security (RLS)
Decision	Shared schema + tenant_id column + PostgreSQL RLS for Release 1 and Release 2, with an explicit path to isolate large/Enterprise-edition tenants into dedicated schemas or databases as a scale/compliance option in Release 3
Rationale	Database-per-tenant does not scale operationally to the thousands-of-tenants target (Vol. 1 Section 2.4 goal) without heavy automation investment the team does not yet have; schema-per-tenant hits connection-pool and migration-fanout limits at similar scale. Shared schema with RLS gives strong logical isolation enforced at the database layer (not just application code), keeps a single migration to run per release, and is the pattern most SaaS platforms at this scale converge on. The Release 3 escape hatch (dedicated schema/database per large tenant) preserves the option for enterprise customers with contractual data-isolation requirements without forcing that cost onto every tenant.
Consequences	Every table must include a non-nullable tenant_id column and an RLS policy from the very first migration (Vol. 4 to formalize); the application's database connection must set the current tenant context (e.g. via a session variable) on every request so RLS policies apply automatically - this is enforced by shared middleware, not per-query discipline.

11.2 Tenant Context Propagation

Request lifecycle - tenant context

```
sequenceDiagram
    participant Client
    participant GW as API Gateway
    participant MW as Tenant Context Middleware
    participant DB as PostgreSQL (RLS enabled)

    Client->>GW: Request with Bearer JWT (tenantId claim)
    GW->>MW: Validate JWT, extract tenantId
    MW->>DB: SET app.current_tenant_id = '...' (per-connection/session)
    MW->>MW: Attach tenantId to correlationId-scoped request context
    MW->>DB: Application query (no manual WHERE tenant_id=... needed)
    DB-->>MW: RLS automatically filters rows to current_tenant_id
    MW-->>Client: Response
```

12. Performance & Scalability Targets

Derived from Vol. 1 Section 2.4 business goals; Volume 7 (Infrastructure) will size the deployment to meet these, and Volume 8 (Testing & QA) will define the load-test suite that verifies them.

Metric	Target	Scope
API P95 latency	< 300ms	Core CRUD operations (invoice create, bill create, journal query) under normal load
API P95 latency (reporting)	< 2s	Custom report generation against the OpenSearch index (Section 2.6)
Event processing lag	< 5s P95	Time from domain event publish to downstream consumer completion (e.g. InvoiceSent to JournalPosted)
Concurrent tenants supported (Release 1 target)	1,000+	Shared-schema RLS model (Section 11), single-region deployment
Concurrent tenants supported (Release 3 target)	10,000+	With Release 3 tenant-isolation escape hatch for largest accounts (Section 11)
Background job processing	Recurring invoice batch for 10,000 tenants completes within a 1-hour nightly window	Worker pool horizontal scaling (Section 9)

13. Traceability & Next Phase

13.1 Traceability to Volumes 1 and 2

Vol. 3 Artifact	Traces To
Section 3 Folder Structure	Vol. 2 Section 3.2 Bounded Context Catalog (CTX-01...CTX-16)
Section 4 Coding Standards CS-01/CS-02	Vol. 2 Section 2.1 modular monolith decision, Section 7 module dependency graph
Section 8 Event Bus Topics	Vol. 2 Section 10 Domain Event Catalog
Section 10 Feature Flags	Vol. 1 Section 9 Editions & Packaging Strategy
Section 11 Multi-Tenancy	Vol. 1 Principle P1
Section 12 Performance Targets	Vol. 1 Section 2.4 Business Goals & Success Measures

13.2 Open Questions for Sign-off

1. Confirm cloud provider preference (AWS/GCP/Azure) if one exists - this affects which managed services (Kafka, Redis, OpenSearch, object storage) Volume 7 will size and price.
2. Confirm whether Nx or Turborepo is preferred for the monorepo tooling, or whether the team has an existing preference/expertise to weigh in.
3. Confirm the Release 3 tenant-isolation escape hatch (Section 11.1) is an acceptable answer to any known enterprise-customer data-residency requirements, or whether isolation needs to be available earlier.

13.3 Next Phase

On approval, Phase 4 begins: Data Architecture (Volume 4) - entity-relationship diagrams, physical schema design (including the tenant_id/RLS pattern from Section 11), indexing and partitioning strategy for high-volume tables, and the migration strategy, all built on the module boundaries (Vol. 2) and stack decisions (this volume).